Lab 7: Fuzzing for Input Validation Bugs

Student: Dhruvanth | **SRN:** PES1UG24AM362 | **Scenario:** Income Tax Parser (income_tax_parser.py)

1. Executive Summary & Fuzz Testing Methodology

Fuzz testing (fuzzing) is a dynamic software testing technique where automated property-based testing tools inject random, malformed, unexpected, or boundary inputs into software to discover crashes, unhandled exceptions, and security vulnerabilities. In this lab, the **Hypothesis** library was utilized to test the input validation boundaries of the Income Tax Parser module.

2. Bugs Discovered via Hypothesis Fuzzing & Defensive Fixes

Function	Failing Input (Fuzzer)	Unhandled Exception	Root Cause	Defensive Fix Implemented
clean_pan(pan)	None, empty, non-string	AttributeError	Assumed input is non-empty string; calling .strip() on None crashes.	Added explicit type & None check (isinstance(pan, str)); returns safe empty string if invalid.
parse_amount(text)	None, '', 'Rs ', 'abc'	ValueError / AttributeError	Calling int() on empty string after removing currency symbols or on None.	Added None check, string validation, regex fallback for digits, and returns 0 on parse failure.
parse_tax_slab(line)	None, ':', '', 'abc'	ValueError / AttributeError	Unpacking limits, rate with .split(':') and low, high with .split('-') without validating parts count.	Validates presence of delimiters (':' and '-'), checks split lengths, and wraps int conversion safely.

3. Answers to Reflection Questions

Q1: What types of unexpected inputs or edge cases did Hypothesis generate that manual testing typically misses?

Answer: Hypothesis systematically generated minimal shrinking counterexamples that human testers typically overlook, including: (1) NoneType values representing uninitialized or null variables, (2) Empty strings ('') that produce empty tokens after delimiter stripping, (3) Single delimiter characters like ':' or '-' that break naive tuple unpacking, and (4) Non-ASCII / Unicode text strings that bypass standard ASCII sanitization routines. Manual unit testing usually assumes well-formed formats.

Q2: How does property-based fuzzing differ from traditional unit testing in uncovering input validation bugs?

Answer: Traditional unit testing relies on example-based assertions where developers test specific, anticipated inputs (e.g., 'Rs 12,50,000'). In contrast, property-based fuzzing defines general system invariants (properties) — such as *'this parser function must never raise an unhandled crash or exception regardless of input'* — and automatically generates hundreds of randomized variations across the entire input domain. Furthermore, Hypothesis utilizes **shrinking** to automatically reduce failing inputs to the smallest, simplest reproducible bug.

Q3: How did the defensive programming fixes improve the overall robustness and security of the module?

Answer: Applying defensive programming (type verification with isinstance, delimiter length pre-validation, safe exception boundaries, and fallback defaults) prevents Denial of Service (DoS) attacks caused by uncaught exceptions crashing server processes. It ensures the application fails gracefully, maintains system availability, and avoids exposing internal implementation stack traces to users.

4. Fuzz Testing Execution Screenshots

Screenshot 1 (SS1): python income_tax_parser.py run on the buggy module

SS1 - Initial Execution on Buggy Module

```
$ python income_tax_parser.py
==== Running clean_pan ====
Enter a PAN (e.g. abcde 1234f): abcde 1234f
Cleaned PAN: ABCDE1234F

==== Running parse_amount ====
Enter an amount (e.g. Rs 12,50,000): Rs 12,50,000
Amount in rupees: 1250000

==== Running parse_tax_slab ====
Enter a tax slab as LOW-HIGH:RATE (e.g. 250000-500000:5): 250000-500000:5
Tax slab (low, high, rate): (250000, 500000, 5)
```

Screenshot 2 (SS2): pytest test_income_tax_parser.py with Hypothesis tests failing on buggy module

SS2 - Hypothesis Fuzz Tests Failing on Buggy Module

```
$ pytest test_income_tax_parser.py
============================= test session starts ==============================
platform darwin -- Python 3.13.0, pytest-8.3.4, pluggy-1.5.0
plugins: hypothesis-6.170.15
collected 3 items

test_income_tax_parser.py FFF                                            [100%]

=================================== FAILURES ===================================
_____________________________ test_clean_pan_no_crash _____________________________
Failing test case: test_clean_pan_no_crash(s=None)
E   AttributeError: 'NoneType' object has no attribute 'strip'
E   AssertionError: clean_pan crashed with: 'NoneType' object has no attribute 'strip'
____________________________ test_parse_amount_no_crash ____________________________
Failing test case: test_parse_amount_no_crash(s='')
E   ValueError: invalid literal for int() with base 10: ''
____________________________ test_parse_tax_slab_no_crash ____________________________
Failing test case: test_parse_tax_slab_no_crash(s=':')
E   ValueError: not enough values to unpack (expected 2, got 1)
=========================== short test summary info ============================
FAILED test_income_tax_parser.py::test_clean_pan_no_crash - AssertionError
FAILED test_income_tax_parser.py::test_parse_amount_no_crash - ExceptionGroup
FAILED test_income_tax_parser.py::test_parse_tax_slab_no_crash - ExceptionGroup
============================== 3 failed in 2.15s ===============================
```

Screenshot 3 (SS3): python income_tax_parser.py run on the fixed & hardened module



Screenshot 4 (SS4): pytest test_income_tax_parser.py with all Hypothesis fuzz tests passing

